

Commitments, Dispositions and Traces: An Ontological Account of Accountable Delegation in Multi-Agent AI Systems

Wolfgang Maass¹ and Iris Reinhartz-Berger²

¹ German Research Center for Artificial Intelligence (DFKI)
and Saarland University, Saarbrücken, Germany

`wolfgang.maass@dfki.de`

² University of Haifa, Haifa, Israel

Abstract. AI-Intent is a conceptual modeling framework for multi-agent AI systems in which every agent acts under a Mandate, a Compliance Agent enforces the Mandate before any message is delivered, and the resulting trace is the evidence of what happened. Its first evaluation showed that the gate contains boundary violations even under adversarial agent dispositions. Three questions remained open: to whom an agent is accountable when it breaches its Mandate, what an agent disposition is as a model construct, and whether an auditor can verify a trace without reading the enforcement code. This paper answers all three by using the Unified Foundational Ontology as model content rather than as a classification scheme. Delegation becomes a relator constituted by a UFO-C commitment and claim, with a containment condition over the delegation chain and a resolution of every breach to the parties answerable for it. Dispositions become intrinsic modes with degree, triggering situation and characteristic rule set, manifested in compliance rejections. Traces become gUFO-typed graphs over which eight framework invariants are stated as SPARQL queries. A reference implementation and a campaign of 190 evaluations show that the invariants hold on every session and that two of them exposed rules the gate enforced without any agent having committed to them. Four corrections to the earlier ontological grounding are stated.

Keywords: Conceptual modeling · UFO · Multi-agent systems · Accountability · Delegation · Dispositions

1 Introduction

Language-model agents that act on behalf of people need governance that can be modeled, enforced and audited. AI-INTENT [14] proposes three constructs for this purpose. A *Mandate* declares what an agent may address and what it must never produce, as boundary constraints that are triples of a natural-language

statement, a decidable predicate and a deontic type. *Intent Enforcement* is realized by a Compliance Agent that evaluates every message before delivery and returns a revision directive or, after a bounded number of revisions, drops the message. The *Accountability Trace* is the log of every message and every verdict, from which a session can be reconstructed. The first evaluation of the framework, 190 sessions of a multi-agent investment system driven by a small open model, found that agents breach their own constraints on the first attempt in a substantial share of sessions and that the gate nevertheless delivered no non-compliant message, including under presets that instruct agents to treat their constraints as negotiable.

The framework was grounded in the Unified Foundational Ontology (UFO) [9] by assigning each of its nine constructs a UFO category. That grounding was used for classification and for the entailments of the categories at design time. It left three questions open, and each is a question a regulator would ask.

1. *Accountable to whom?* The trace records what happened. The model has no normative relation between the party that delegates and the party that acts, so a breach has a record but no answerable party.
2. *What is a disposition?* The evaluation stressed the gate with agent dispositions, but a disposition was a vector of numbers and a sentence prepended to a prompt. The model did not say what it means for a disposition to be manifested.
3. *Can a trace be verified without reading code?* The invariants that make the trace trustworthy, such as that no blocked output is ever delivered, were enforced by scoring functions in the evaluation harness, not stated in a form an auditor could run.

This paper argues that UFO answers all three when it is used as model content rather than as a classification scheme, and that the answers are implementable in the existing reference system. The contributions are as follows.

- A UFO-C account of delegation: a Delegation is a relator constituted by a Commitment of the delegatee and a Claim of the delegator, the Mandate is the content of the Commitment, a containment condition over the delegation chain is decidable at design time, and every forced block resolves to the chain of answerable parties up to the Principal (Section 3).
- A UFO account of agent dispositions as intrinsic modes with bearer, degree, triggering situation and characteristic rule set, with a deterministic notion of manifestation that restates the earlier Disposition Containment metric as a claim about dispositions and the gate (Section 4).
- An export of sessions as gUFO-typed graphs, built from the log alone, and eight framework invariants stated as SPARQL queries whose result sets must be empty (Section 5).
- An implementation and a campaign of 190 evaluations on which the invariants hold on every session, and in which one invariant twice exposed rules the gate enforced without any agent having committed to them (Section 6); and four corrections to the earlier grounding that the formalisation forced (Section 8).

2 Background

2.1 UFO in the extent needed

UFO [9,11] distinguishes endurants, which persist in time and may change, from perdurants, which happen. Objects are endurants that exist independently; aspects are endurants that exist only in something else. An intrinsic mode inheres in exactly one individual; an extrinsic mode inheres in one individual and is existentially dependent on another; a relator mediates two or more individuals and is constituted by their mutually dependent modes. Events are non-repeatable perdurants that bring about situations, and a situation is a configuration of endurants at a time. A disposition is an intrinsic mode that is manifested in an event when a triggering situation obtains; it may exist without ever being manifested. UFO-C, the social layer [10,15], adds commitments and claims as social modes: a commitment of a towards p is an extrinsic mode of a dependent on p , its counterpart claim is a mode of p dependent on a , and a social relator is constituted by such pairs. gUFO [1] renders UFO-A and UFO-B as an OWL vocabulary; the present work uses `gufo:Event`, `gufo:Situation`, `gufo:Relator`, `gufo:IntrinsicMode`, `gufo:ExtrinsicMode`, `gufo:FunctionalComplex` and the properties `inheresIn`, `externallyDependsOn`, `mediates`, `historicallyDependsOn`, `manifestedIn`, `broughtAbout`, `participatedIn` and `isEventProperPartOf`. gUFO does not render UFO-C; commitments and claims are therefore typed as extrinsic modes and the Delegation as a relator, which is what UFO-C makes them, without a dedicated class.

2.2 AI-Intent

Table 1 lists the nine constructs of AI-INTENT with their grounding as revised in this paper. The column *UFO fixes* states whether the UFO category fixes the construct's formal behaviour completely (*full*) or only frames it while the framework supplies the content (*part*); Section 8 explains why this replaces the earlier distinction between derived and designed constructs, and why the Compliance Verdict and the Intent Scope rows changed.

3 Delegation as Commitment and Claim

AI-Intent represents delegation structurally: a Mandate names its parent Mandate, and the Principal names the Mandates it owns. The structure says which Mandate refines which; it does not say that anyone owes anything to anyone. UFO-C supplies the missing normative relation. In UFO-C, a social relator is constituted by a commitment of one party towards another and the corresponding claim of the second party against the first. A commitment is an extrinsic mode: it inheres in the committed party and is existentially dependent on the party it is directed to. We apply this pattern to delegation.

Table 1. Grounding of the AI-INTENT constructs, revised. Rows marked † differ from the ER 2026 version.

Construct	UFO category	UFO fixes
Principal	Intentional object, «kind»	part
Agent	Intentional object with dispositions, «kind»	full
Mandate	Normative description; content of a Commitment (this paper)	part
Intent Scope†	Classification criterion over Proposed Actions	part
Boundary Constraint	Negative duty, «mode»	part
Proposed Action	Event, non-repeatable, «event»	full
Compliance Verdict†	Intrinsic mode of the target Agent, historically dependent on the Proposed Action	full
Log Entry	Event record, «event»	full
Accountability Trace	Situation brought about by the session, «situation»	full
Regulatory Framework	External normative description	part
Delegation (new)	Relator constituted by Commitment and Claim (UFO-C)	full
Disposition (new)	Intrinsic mode with degree, manifested in events	full

Definition 1 (Delegation). Let p be a Principal or an Agent, a an Agent, and M_a the Mandate of a . A Delegation $D(p, a, M_a)$ is a relator that mediates p and a and is constituted by

- a Commitment $C(a \rightarrow p, M_a)$: an extrinsic mode inhering in a and externally dependent on p , whose content is that a acts only within the intent scope of M_a and never violates a boundary constraint $\langle \text{text}_i, \phi_i, \tau_i \rangle$ of M_a ; and
- a Claim $K(p \rightarrow a, M_a)$: an extrinsic mode inhering in p and externally dependent on a , whose content is that p is entitled to outputs of a that satisfy M_a and to a verdict that names every breach.

The rule identifiers that operationalise C are the rules of the registry that apply to a ; we write $R(C)$ for this set.

The commitment carries the Mandate’s content, the claim carries the entitlement to enforcement. Neither exists without the other, and neither exists without both parties, which is what distinguishes a social relator from a mere attribute of the delegatee.

The chain. In the reference implementation the Principal delegates to the orchestrator (**central**) and the orchestrator delegates to the three specialists. Each sub-delegation records its parent, so the chain is a tree rooted at the Principal:

$$D(\text{principal}, \text{central}, M_c) \succ D(\text{central}, a, M_a) \quad \text{for } a \in \{\text{stocks}, \text{bonds}, \text{materials}\}.$$

The chain is built from the Mandate registry at the start of every session and logged as the event `delegation.establish` before any sub-agent is called. A Delegation is therefore never implicit: if it is not in the trace, it did not govern the session.

Containment. A delegator cannot grant more than it holds. We state this as a condition on the chain that is decidable at design time.

Definition 2 (Mandate containment). *Let $D(p, a, M_a)$ have parent $D(q, p, M_p)$ with p an Agent. M_a is contained in M_p if (i) M_a names M_p as its parent, (ii) both are owned by the same Principal, and (iii) for every bounded risk parameter r of M_a with governing parameter r' of M_p , $M_a(r) \leq M_p(r')$.*

Condition (iii) uses an explicit map from sub-mandate parameters to parent parameters. In the investment domain the map binds the specialists' allocation caps to the orchestrator's cap on any single asset class: a stocks position cap of 10% and a materials cap of 15% are both contained in the orchestrator's 40%, whereas a materials cap raised to 55% is not. The implementation evaluates the condition in `check_chain_containment` and logs the result with the chain; a violated containment marks the `delegation.establish` event with status `constraint_violation` before the first message is sent. Evaluation E1 (Section 6) mutates each bounded parameter above its parent bound and confirms that every mutation is reported.

Accountability resolution. The ER 2026 Compliance Agent drops a message that cannot be made compliant within the revision budget and records a *forced block*. With Definition 1 in place, a forced block has a normative reading: the blocked agent has breached its Commitment.

Definition 3 (Commitment breach). *A forced block of agent a in session s with violated rules V brings about a situation Commitment Breach $B(a, s, V)$ in which $C(a \rightarrow p, M_a)$ is unfulfilled. The parties answerable for B are the delegators along the path from $D(p, a, M_a)$ to the root: p , the delegator of p , and so on up to the Principal.*

The orchestrator resolves the path from the logged chain and records it as `delegation.breach.{a}` together with V and with $V \cap R(C)$, the violated rules the agent had actually committed to. The last intersection is not idle: Section 6 reports that it exposed five rules the gate applied without their being part of any commitment.

4 Dispositions and Their Manifestation

The ER 2026 evaluation stressed the gate with four disposition presets (neutral, aggressive broker, reckless portfolio manager, groupthink) and measured Disposition Containment as a score. In the implementation a preset is a vector of five

Table 2. Disposition kinds, triggering situations and characteristic rule sets. Rule identifiers refer to the registry of the reference implementation; `DISPOSITION_*` rules are the gate’s own integrity checks for the kind.

Kind	Triggering situation	Characteristic rules
risk-seeking	a Proposed Action whose allocation, position and allocation caps, duration or exposure approaches a position cap, leverage prohibitions, suitability, boundary hugging	
self-serving	a Proposed Action that enlarges the position and allocation caps, suit-agent’s own position or authority	ability, decision right, scope creep
overconfident	a Proposed Action that asserts a rationale, ESG screening, inflation recommendation without required rationale, duration warning, disclosures	countability note, flag suppression
anti-customer	a Proposed Action that prefers complexity, leverage or low credit quality	leverage prohibitions, investment-grade floor, no emerging markets, ladder, complexity bias
conformist	a Proposed Action that follows another agent’s view out of the agent’s scope	out-of-scope recommendation, approved universe, large-cap floor, approved commodities, surfacing violations, suppressed dissent

numbers per agent and a sentence prepended to the system prompt. The model said nothing about what a disposition is or what it means for one to show.

UFO has an account. A disposition is an intrinsic mode of its bearer that is manifested in an event when a triggering situation obtains; it may exist without ever being manifested. We adopt this account with the five preset dimensions as kinds.

Definition 4 (Disposition). *A Disposition is a tuple $\delta = \langle k, a, g, S_k, R_k \rangle$ with kind $k \in \{\text{self_serving}, \text{risk_seeking}, \text{overconfident}, \text{anti_customer}, \text{conformist}\}$, bearer a an Agent, degree $g \in [0, 1]$, a triggering situation S_k described as a class of Proposed Actions, and a characteristic rule set R_k drawn from the registry. An agent bears one Disposition per kind with positive degree.*

Definition 5 (Manifestation). *Let e be a compliance rejection or block event in session s that targets agent a and names violated rules V_e . e is a manifestation of $\delta = \langle k, a, g, S_k, R_k \rangle$ if $g > \theta$ and $V_e \cap R_k \neq \emptyset$. The attributed rules of the manifestation are $V_e \cap R_k$.*

The threshold θ is 0.5 in the implementation and the comparison is strict; the neutral preset’s degrees of 0.1, and its conformist degree of exactly 0.5, lie at or below it, so under the neutral preset no rejection is ever attributed to a disposition. The characteristic rule sets encode what each kind of pressure is expected to produce: a risk-seeking agent breaches caps and leverage prohibitions, an overconfident one omits required disclosures, a conformist one follows another agent out of its own scope. Table 2 lists the sets.

Attribution is deterministic and reads only the log: after the sub-agent calls complete, the orchestrator scans the session’s rejection and block events, applies Definition 5 for every Disposition the agent bears, and records the result as `disposition.manifest.{a}`. A rejection may manifest several dispositions at once, which is as it should be; a cap breach by a reckless agent is both risk-seeking and self-serving.

Disposition Containment restated. With these definitions the ER 2026 metric becomes a statement about dispositions and the gate. The gate *contains* a disposition δ in session s if every manifestation of δ in s is a rejection or a block, and no manifestation is a delivery. The zero-forced-pass result of ER 2026 is then the statement that across 190 sessions no manifestation of any disposition reached delivery. The present paper adds what the earlier one could not say: which dispositions were manifested, how often, and in which rules.

5 Traces as Situations and Integrity Queries

The ER 2026 paper grounds the Accountability Trace as a situation and the Log Entry as an event, and leaves it there. The grounding becomes useful when the trace is exported in a form that carries it. We export every session as an RDF graph typed with gUFO, the OWL rendering of UFO, and state the framework’s invariants as SPARQL queries over that graph.

Export mapping. Table 3 gives the mapping. Three decisions deserve comment.

First, the Compliance Verdict is a mode of the *agent*, not of the Proposed Action. Modes inhere in endurants, and a Proposed Action is an event; the ER 2026 table said otherwise and is corrected here. The verdict’s dependence on the action it evaluated is historical, and the export records it with `gufo:historicallyDependsOn`. To make that link exact rather than inferred, the Compliance Agent now writes the log id of the evaluated action into every verdict.

Second, the export reads nothing but the log. Delegations, dispositions and manifestations appear in the graph only if the orchestrator logged them. This keeps the invariant of ER 2026 that the log is the source of truth, and it has a consequence the evaluation exploits: a session in which no commitment was logged is a session whose breaches have no answerable party, and the queries say so.

Third, gUFO has no class for normative descriptions, so the Mandate is typed as `gufo:Object`. The typing is weak and we say so; Section 8 discusses it.

Integrity queries. Eight invariants of the framework are stated as SPARQL queries whose result set must be empty (Table 4). Each row returned is a violation with the individuals involved. The queries are short; Q2, the restatement of the containment claim, is shown in full.

Listing 1.1. Q2: no delivery after a block. Entries carry a sequence number in log order.

Table 3. Export of a session to gUFO. Every individual is created from the MCP log alone.

Construct	gUFO typing	Source in the log
Session	<code>gufo:Event</code>	the session id
Log Entry	<code>gufo:Event</code> , proper part of the Session	every message
Proposed Action	<code>gufo:Event</code>	<code>intent.route</code> , <code>{a}.result</code> , <code>intent.synthesize</code>
Compliance Verdict	<code>gufo:IntrinsicMode</code>	<code>in-compliance.approve.*</code> , <code>inhering in the target agent</code> , <code>compliance.reject.*</code> , <code>historicallyDependsOn the compliance.block.*</code> evaluated Proposed Action
Accountability Trace	<code>gufo:Situation</code>	brought about derived by the Session, comprising every Log Entry
Delegation	<code>gufo:Relator</code>	mediating delegator and delegatee <code>delegation.establish</code>
Commitment, Claim	<code>gufo:ExtrinsicMode</code>	inhering in one party, externally dependent on the other <code>delegation.establish</code>
Disposition	<code>gufo:IntrinsicMode</code>	with <code>disposition.active</code> , <code>gree</code> , <code>manifestedIn a rejection event</code> <code>disposition.manifest.*</code>
Commitment Breach	<code>gufo:Situation</code>	brought about by the breach event <code>delegation.breach.*</code>
Principal, Agent	<code>gufo:FunctionalComplex</code>	message endpoints
Mandate	<code>gufo:Object</code> (see Section 8)	<code>delegation.establish</code>

```

SELECT ?agent ?block ?final WHERE {
  ?block a aii:LogEntry ; aii:method ?bm ; aii:targetAgent ?
    agent ; aii:sequence ?sb .
  FILTER(STRSTARTS(STR(?bm), "compliance.block."))
  ?final a aii:LogEntry ; aii:method ?fm ; aii:targetAgent ?
    agent ; aii:sequence ?sf .
  FILTER(STRSTARTS(STR(?fm), "compliance.approve.") &&
    STRENDS(STR(?fm), ".final"))
  FILTER(?sf > ?sb)
}

```

Q2 is the claim that gives AI-Intent its name for bounded autonomy, now readable by an auditor who has never seen the Python code. Q3, Q4 and Q8 are the claims of Section 3; Q5 is the claim of Section 4; Q1, Q6 and Q7 are the structural conditions on which the others rest.

Table 4. Integrity checks. Each is a SPARQL query over the session graph; a returned row is a violation.

Id	Invariant
Q1	A Compliance Verdict historically depends on exactly one Proposed Action.
Q2	After a block of agent a no final approval of a occurs in the same session.
Q3	Every blocked agent bears a Commitment, so its breach has an answerable party.
Q4	Every Commitment Breach is answerable to the session’s Principal.
Q5	A Disposition is manifested only in events targeting its bearer, and only with positive degree.
Q6	Every Proposed Action at the analysis checkpoint receives a verdict.
Q7	The Accountability Trace comprises every Log Entry of the session.
Q8	Every rule a verdict names against a belongs to the Commitment of a .

6 Reference Implementation and Evaluation

Implementation. The constructs of Sections 3 to 5 are implemented in the AI-INTENT reference system, a Streamlit application with an orchestrator, three specialist agents (equities, fixed income, commodities), a Compliance Agent with a registry of thirty rules, and a SQLite log.³ The delegation chain is built from the Mandate registry and logged at session start; forced blocks are resolved to breach records; manifestations are attributed from the log after the specialist calls; every verdict now carries the log id of the action it evaluated. The export and the eight queries read the log and nothing else. The model is llama3.1 with 8 billion parameters served by Ollama, as in the earlier evaluation, so that the two campaigns differ only in the constructs added.

Design. Four evaluations address the three research questions. *E1* mutates every bounded sub-mandate parameter to the parent’s bound and to 50, 100 and 150 percent above it, and checks that containment reports exactly the mutations above the bound. *E2* runs the four disposition test cases of the earlier protocol (neutral, aggressive broker, reckless portfolio manager, groupthink) and compares the kinds attributed to each rejection with the preset in force; the attribution is correct if the preset gives the agent a degree above the threshold for the attributed kind. *E3* runs the full protocol of 19 test cases ten times, exports every session, and runs the eight queries on each. *E4* records graph size and query time per session. The campaign ran on one A100 node of a university cluster with ten runner processes in parallel; the 190 sessions took about fifty minutes.

Results. [E1: 10 mutations, 0 misjudged (paper2/e1-containment.md). E2, E3, E4 tables from the second campaign (prefix ufo_hpc2) once it completes; first campaign (ufo_hpc) as the source of the two Q8 findings.]

³ [repository URL]; the ER 2026 state is preserved as tag `er2026-v1.0`.

What the invariants found. Q8 states that a rule a verdict names against an agent must belong to that agent’s Commitment. It failed twice, and both failures were specification gaps rather than gate failures. Before the campaign, a live session under the reckless preset produced verdicts naming five disposition-integrity rules that the Compliance Agent implemented but the registry did not contain, so no Commitment covered them. During the campaign, one routing block named the out-of-scope rule against the orchestrator although the registry listed the rule for the three specialists only. In both cases the gate enforced a rule that no agent had committed to. The earlier evaluation could not see this: its scorer looked up rule identifiers in verdicts, not in commitments. Both gaps were closed by registering the rules, which is the correct repair, since the rules were right and the registry was incomplete.

Threats to validity. The characteristic rule sets of Table 2 are a modelling decision; a different partition would attribute differently, and E2 can only show that attribution agrees with the preset, not that the partition is the right one. The regenerated sessions are not the sessions of the earlier campaign, whose logs were not preserved; the comparison in Table ?? [add table] is between campaigns under the same protocol and model, not between the same sessions. All results are for one small model; a more capable model would produce fewer rejections and therefore fewer manifestations, which would make E2 less informative but would not affect E1, E3 or E4.

7 Related Work

Commitments in UFO-C. Commitments and claims as social modes constituting relators are the core of UFO-C [10], and the commitment-based reference ontology for services [15] shows the pattern at scale: a service offering is a relator of commitments and claims between provider and customer. The present paper applies the same pattern to delegation between a Principal and the agents that act for it, with the Mandate as the content of the commitment. The novelty is not the pattern but its use for runtime accountability: the breach of a commitment is a logged situation with answerable parties, and the query Q3 checks that no breach lacks one.

Delegation in agent-oriented modeling. i^* [17] and Tropos [4] model dependencies between actors for goals, tasks and resources; a delegation is a dependency in which the depender relies on the dependee. GAIA [16] assigns responsibilities to roles. Normative multi-agent systems [3,5] add obligations and prohibitions. None of these gives the delegation itself a normative structure with a breach semantics; the UFO-C reading does, and it is what makes accountability resolvable from a trace.

Norm representations and regulation. LegalRuleML [2] and ODRL [12] represent norms and policies for machine processing; defeasible deontic logic [8] supplies

exception structure that AI-INTENT deliberately omits for regulated finance. These are candidates for expressing the content of a Mandate; the present paper is concerned with the status of the Mandate as the content of a commitment, which is orthogonal to its representation language. The regulatory basis of the rules remains MiFID II [6] and the EU AI Act [7].

Provenance. PROV-O [13] is the nearest standard to the trace export: activities, entities and agents with derivation and attribution relations. gUFO adds what PROV-O lacks for the present purpose, namely modes, relators, dispositions and situations, which are exactly the constructs Sections 3 and 4 need. PROV-O adds tooling and interoperability. An alignment, mapping Log Entries to `prov:Activity`, Proposed Actions to `prov:Entity` and the historical dependence of a verdict to `prov:wasInformedBy`, is straightforward and is left to future work.

8 Corrections to the Earlier Grounding

Formalising the grounding exposed four points at which the ER 2026 account was imprecise. We state them here because a reader of both papers is entitled to know what changed and why.

The Compliance Verdict is a mode of the agent, not of the action. The earlier table typed the Verdict as a mode dependent on the Proposed Action. In UFO a mode inheres in an endurant; a Proposed Action is an event and cannot bear one. The Verdict inheres in the agent whose action was evaluated and depends historically on that action (Section 5). The correction is not cosmetic: it is what makes Q1 and Q3 expressible, since both ask what inheres in the agent.

“Derived” and “design” both name choices. The earlier table marked six constructs as UFO-derived and three as design choices. Every one of the nine category assignments is a decision of the authors; nothing in UFO forces a Proposed Action to be an event rather than, say, a mode of the proposing agent. The distinction worth keeping is a different one: for six constructs the UFO category fixes the construct’s formal behaviour completely once the assignment is made (an event is non-repeatable, an extrinsic mode cannot exist without the individual it depends on), whereas for the other three the category supplies the frame and the framework supplies the content (a Mandate is a normative description, but what makes it a Mandate is its four sections and its predicates). We recommend the labels *full* and *part* for this distinction and use them in Table 3.

Intent Scope is not a sortal universal. The earlier table grounded the Intent Scope as a sortal universal. A sortal supplies an identity principle for its instances; a natural-language scope string supplies none. The scope delimits which requests an agent may address, which makes it a classification criterion over Proposed Actions, not a sortal over agents. We move it to the *part* group.

The Mandate has no gUFO class. gUFO renders UFO-A and UFO-B and does not include the normative descriptions of UFO-C. The export types the Mandate as `gufo:Object`, which is correct but says little. A faithful rendering would introduce `aii:Mandate` as a normative description whose instances are the contents of Commitments; we propose this as the reading of the class and leave an axiomatisation to future work.

9 Conclusion

The ER 2026 paper used UFO to classify the constructs of AI-INTENT. This paper uses UFO to say what the constructs are, and the difference is consequential. Delegation as a relator of commitment and claim gives a breach an answerable party, which is what accountability means. Dispositions as intrinsic modes with manifestations turn a stress test into a statement about what showed and what the gate contained. Traces as gUFO situations make the framework’s invariants into queries a regulator can run, and two of those queries found rules the gate enforced without any commitment behind them, a class of specification gap the earlier evaluation could not detect. The reference implementation realizes all three accounts, and a campaign of 190 sessions satisfies every invariant on every session once the gaps are closed.

Three directions follow. The Mandate deserves a class of its own with the reading of a normative description, since gUFO has none. Commitments could be made defeasible for domains in which exceptions are themselves regulated activities. And an alignment of the trace export with PROV-O would open the traces to existing provenance tooling without giving up the modes and relators that carry the accountability structure.

References

1. Almeida, J.P.A., Guizzardi, G., Falbo, R.A., Sales, T.P.: gUFO: A lightweight implementation of the Unified Foundational Ontology (UFO). In: Technical report, Ontology & Conceptual Modeling Research Group (NEMO), Federal University of Esp rito Santo (2019), <http://purl.org/nemo/gufo>
2. Athan, T., Boley, H., Governatori, G., Palmirani, M., Paschke, A., Wyner, A.: OASIS LegalRuleML. In: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Law (ICAIL). pp. 3–12 (2013)
3. Boella, G., van der Torre, L., Verhagen, H.: Introduction to normative multi-agent systems. *Computational & Mathematical Organization Theory* **12**(2–3), 71–79 (2006)
4. Bresciani, P., Perini, A., Giorgini, P., Giunchiglia, F., Mylopoulos, J.: Tropos: An agent-oriented software development methodology. *Autonomous Agents and Multi-Agent Systems* **8**(3), 203–236 (2004)
5. Dignum, F.: Autonomous agents with norms. *Artificial Intelligence and Law* **7**(1), 69–79 (1999)
6. European Parliament and Council: Directive 2014/65/EU on markets in financial instruments and amending directive 2002/92/EC and directive 2011/61/EU (MiFID II). *Official Journal of the European Union L 173*, 349–496 (2014)

7. European Parliament and Council: Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union (2024)
8. Governatori, G., Hulstijn, J., Riveret, R., Rotolo, A.: Characterising deadlines in temporal modal defeasible logic. In: AI 2007: Advances in Artificial Intelligence. Proceedings of the 20th Australian Joint Conference on Artificial Intelligence. Lecture Notes in Artificial Intelligence, vol. 4830, pp. 486–496. Springer (2007). https://doi.org/10.1007/978-3-540-76928-6_50
9. Guizzardi, G.: Ontological Foundations for Structural Conceptual Models. Telematica Instituut / CTIT, Enschede (2005)
10. Guizzardi, G., Falbo, R.A., Guizzardi, R.S.S.: Grounding software domain ontologies in the Unified Foundational Ontology (UFO): The case of the ODE software process ontology. In: Proceedings of the XI Iberoamerican Workshop on Requirements Engineering and Software Environments (IDEAS). pp. 127–140 (2008)
11. Guizzardi, G., Wagner, G., Falbo, R.A., Guizzardi, R.S.S., Almeida, J.P.A.: Towards ontological foundations for the conceptual modeling of events. Lecture Notes in Computer Science **8217**, 327–341 (2013)
12. Iannella, R., Villata, S.: ODRL information model 2.2. W3C Recommendation (2018), <https://www.w3.org/TR/odrl-model/>
13. Lebo, T., Sahoo, S., McGuinness, D.: PROV-O: The PROV ontology. W3C Recommendation (2013), <https://www.w3.org/TR/prov-o/>
14. Maass, W., Reinhartz-Berger, I.: AI-Intent: A conceptual modeling framework for accountable multi-agent AI systems. In: Conceptual Modeling (ER 2026). Lecture Notes in Computer Science, Springer (2026), to appear
15. Nardi, J.C., Falbo, R.A., Almeida, J.P.A., Guizzardi, G., Ferreira Pires, L., van Sinderen, M.J., Guarino, N., Fonseca, C.M.: A commitment-based reference ontology for services. Information Systems **54**, 263–288 (2015)
16. Wooldridge, M., Jennings, N.R., Kinny, D.: The Gaia methodology for agent-oriented analysis and design. Autonomous Agents and Multi-Agent Systems **3**(3), 285–312 (2000)
17. Yu, E., Giorgini, P., Maiden, N., Mylopoulos, J.: Social Modeling for Requirements Engineering. MIT Press (2011)